

Remove Nth Node From End of Linked List

Structured DSA Study Notebook

This notebook follows the uploaded TUF transcript. It covers the brute-force length-counting approach first and then the optimized fast-and-slow pointer approach. ■filecite■turn4file0■L18-L74■

1. Introduction

The problem gives the head of a singly linked list and an integer n . The task is to remove the n th node counted from the end and return the head of the modified list. If the removed node is the original head, the next node becomes the new head. ■filecite■turn4file0■L30-L106■

2. Problem Statement

Input:
head -> 1 -> 2 -> 3 -> 4 -> 5 -> NULL
 $n = 2$

Count from the end:
5 -> 1st
4 -> 2nd

Delete:
4

Output:
1 -> 2 -> 3 -> 5 -> NULL

3. Required Concepts

3.1 Singly Linked List Traversal

```
ListNode temp = head;

while(temp != null){

    // Process current node

    temp = temp.next;
}
```

3.2 Previous Node Is Needed

To delete a node from a singly linked list, the useful pointer is the node immediately before the node being deleted, because its next link must be changed. The transcript emphasizes reaching the previous node rather than only reaching the node to delete. ■filecite■turn4file0■L182-L264■

Before:
previous -> deleteNode -> next

After:
previous -> next

4. Approaches

The transcript presents two approaches: a brute-force two-traversal method based on calculating the list length, followed by an optimized two-pointer method that keeps the required distance between fast and slow. ■filecite■turn4file0■L826-L850■

5. Approach 1 — Brute Force Using Length

5.1 Idea

1. Find the length of the linked list.
2. If n equals the length, delete the head.

3. Otherwise calculate:
 result = length - n
4. Traverse to the previous node.
5. Delete the required node.
6. Return head.

The transcript first counts the complete list, then uses length - n to locate the previous node of the node that must be deleted. ■filecite■turn4file0■L134-L264■

5.2 Pseudocode

```

START

count = 0
temp = head

WHILE temp != NULL
    count++
    temp = temp.next
END WHILE

IF count == n
    RETURN head.next
END IF

result = count - n
temp = head

WHILE temp != NULL
    result--

    IF result == 0
        BREAK
    END IF

    temp = temp.next
END WHILE

deleteNode = temp.next

temp.next = temp.next.next

RETURN head

END

```

5.3 Original Core Code From Transcript

```

// Count list length
counter = 0;
temporary = head;

while(temporary != null){

    counter++;
    temporary = temporary.next;
}

// Head deletion case
if(counter == n){
    head = head.next;
    return head;
}

// Find previous node
resultant = counter - n;
temporary = head;

while(temporary != null){

    resultant--;

    if(resultant == 0){
        break;
    }
}

```

```

    }

    temporary = temporary.next;
}

// Delete node
deleteNode = temporary.next;

temporary.next = temporary.next.next;

return head;

```

These steps follow the transcript's length calculation, head-deletion case, previous-node traversal, and link change.

■filecite■turn4file0■L490-L730■

5.4 Complete Java Code

```

class Solution {

    public ListNode removeNthFromEnd(ListNode head, int n) {

        // Count the length of the list
        int count = 0;

        // Start traversal from head
        ListNode temp = head;

        // Traverse the complete list
        while(temp != null){

            // Increase the length
            count++;

            // Move to the next node
            temp = temp.next;
        }

        // If nth node from end is the head
        if(count == n){

            // Move head to the next node
            head = head.next;

            // Return the new head
            return head;
        }

        // Number of steps needed to reach previous node
        int resultant = count - n;

        // Start again from head
        temp = head;

        // Find the previous node
        while(temp != null){

            // Reduce the remaining position
            resultant--;

            // Stop at the previous node
            if(resultant == 0){
                break;
            }

            // Move to the next node
            temp = temp.next;
        }

        // Store the node to be deleted
        ListNode deleteNode = temp.next;

        // Skip the node to be deleted
        temp.next = temp.next.next;
    }
}

```

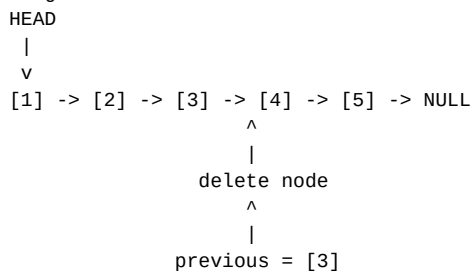
```

        // Return the original head
        return head;
    }
}

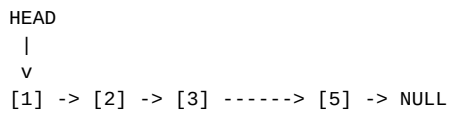
```

5.5 Static Visual Representation

Original:



After:



5.6 Complexity

Time:

$O(n)$

Space:

$O(1)$

Reason:

The list is traversed to find its length
and traversed again to reach the previous node.

6. Approach 2 — Optimized Fast and Slow Pointers

6.1 Intuition

The brute-force method calculates the length explicitly. The optimized idea avoids that by moving fast n steps ahead, then moving fast and slow together. When fast reaches the end, slow is positioned at the previous node of the node to delete. ■filecite■turn4file0■L830-L1006■

6.2 Pointer Setup

Example:

```
1 -> 2 -> 3 -> 4 -> 5 -> NULL
^
|
fast
```

Move fast n steps ahead.

Then move fast and slow together.

The required position is found without calculating the length.

The transcript describes moving fast n steps first and then moving fast and slow simultaneously.

■filecite■turn4file0■L922-L1006■

6.3 Pseudocode

START

fast = head

```
FOR i = 0 TO n - 1
    fast = fast.next
END FOR
```

```
IF fast == NULL
    RETURN head.next
END IF
```

slow = head

```
WHILE fast.next != NULL
    fast = fast.next
    slow = slow.next
END WHILE
```

deleteNode = slow.next

slow.next = slow.next.next

RETURN head

END

6.4 Original Core Code From Transcript

```
// Fast pointer
fast = head;

// Move fast n steps
for(i = 0; i < n; i++){
    fast = fast.next;
}

// Slow pointer
slow = head;

// Move both pointers
while(fast.next != null){
    slow = slow.next;
    fast = fast.next;
}
```

```

}

// Store node to delete
deleteNode = slow.next;

// Skip deleted node
slow.next = slow.next.next;

// Return head
return head;

```

The transcript then handles the important case where fast reaches null after the n-step movement, meaning the head must be removed. ■filecite■turn4file0■L1211-L1298■

6.5 Complete Java Code — Line by Line

```

class Solution {

    public ListNode removeNthFromEnd(ListNode head, int n) {

        // Start fast pointer at head
        ListNode fast = head;

        // Move fast pointer n nodes ahead
        for(int i = 0; i < n; i++){

            // Move fast one node
            fast = fast.next;
        }

        // If fast reaches null,
        // the head is the node to delete
        if(fast == null){

            // Return the second node as the new head
            return head.next;
        }

        // Start slow pointer at head
        ListNode slow = head;

        // Move both pointers until fast reaches the last node
        while(fast != null && fast.next != null){

            // Move fast one node
            fast = fast.next;

            // Move slow one node
            slow = slow.next;
        }

        // Store the node to be deleted
        ListNode deleteNode = slow.next;

        // Skip the node to be deleted
        slow.next = slow.next.next;

        // Return the original head
        return head;
    }
}

```

6.6 Important Java Condition

Correct:

```

while(fast != null && fast.next != null){

    fast = fast.next;
    slow = slow.next;
}

```

Wrong:

```

while(fast.next != null && fast != null){

```

```

    // fast.next is accessed before fast is checked
}

```

Java evaluates the left side of && first. Therefore fast must be checked before accessing fast.next.

6.7 Static Visual Representation

Example:

```

1 -> 2 -> 3 -> 4 -> 5 -> NULL
n = 2

```

After moving fast n steps:

```

slow
|
v
1 -> 2 -> 3 -> 4 -> 5 -> NULL
                ^
                |
                fast

```

Move both until fast reaches the last node:

```

1 -> 2 -> 3 -> 4 -> 5 -> NULL
        ^           ^
        |           |
        slow        fast

```

slow.next is the node to delete.

After:

```

1 -> 2 -> 3 -----> 5 -> NULL

```

6.8 Head Deletion Edge Case

Example:

```

1 -> 2 -> 3 -> 4 -> 5 -> NULL
n = 5

```

Move fast 5 steps:

```

fast = NULL

```

Therefore:

head is the node to delete.

Return:

```

head.next

```

Result:

```

2 -> 3 -> 4 -> 5 -> NULL

```

The transcript explicitly uses this condition: if fast reaches null after the initial n-step movement, return head.next.

■filecite■turn4file0■L1211-L1298■

7. Edge Cases

Case 1: Empty list

```

head = NULL

```

Return head

Case 2: One node

```

1 -> NULL

```

```

n = 1

```

Return NULL

Case 3: Delete head

```

1 -> 2 -> 3 -> 4 -> 5

```

```

n = 5

```

Return 2 -> 3 -> 4 -> 5

Case 4: Delete tail

```

1 -> 2 -> 3 -> 4 -> 5

```

```
n = 1
Delete 5
```

```
Case 5: Delete middle
1 -> 2 -> 3 -> 4 -> 5
n = 2
Delete 4
```

8. Complexity Comparison

Approach	Main Work	Time	Extra Space
Brute Force	Length + second traversal	$O(n)$	$O(1)$
Fast/Slow	n-gap + simultaneous traversal	$O(n)$	$O(1)$

The transcript gives $O(n)$ time and $O(1)$ space for the optimized approach. ■filecite■turn4file0■L1334-L1404■

9. Quick Revision

Brute Force:

1. Count list length.
2. If count == n, delete head.
3. Otherwise result = count - n.
4. Reach previous node.
5. Change previous.next.
6. Return head.

Optimized:

1. fast = head.
2. Move fast n steps.
3. If fast == null, return head.next.
4. slow = head.
5. Move fast and slow together.
6. Stop when fast is at the last node.
7. deleteNode = slow.next.
8. slow.next = slow.next.next.
9. Return head.

Complexity:

Time = $O(n)$
Space = $O(1)$

10. Conclusion

The key optimization is to maintain an n-node gap between fast and slow. This removes the need to calculate the list length explicitly and places slow at the previous node of the node that must be removed.

■filecite■turn4file0■L986-L1006■